

1. 已提交

以下哪个关键字用于声明一个类模板？

(B)

A. virtual

B. template

C. class

D. static

得分：3.00

2. 已提交

函数模板特化的主要目的是？

(B)

A. 提高运行时性能

B. 为特定类型提供定制实现

C. 避免编译器类型推导

D. 增加代码可读性

得分：3.00

3. 已提交

在继承中，派生类可以通过以下哪种方式访问基类的保护成员？

(A)

A. 直接访问

B. 仅通过基类对象访问

C. 仅通过友元函数访问

D. 无法访问

得分：3.00

4. 已提交

以下哪个运算符不能被重载？

(C)

A. ==

B. \*

C. sizeof

D. []

得分：3.00

5. 已提交

在C++中，关于析构函数的说法，哪项是错误的？

B

A. 析构函数可以是虚函数

B. 析构函数可以有参数

C. 析构函数没有返回值

D. 析构函数名称与类名相同

得分：3.00

填空题

1. 已提交

在多重继承中，解决菱形继承问题的方法是使用 虚 继承

得分：3.00

2. 已提交

重载二元运算符（如+）作为成员函数时，函数签名通常有 1 个显式参数

得分：3.00

3. 已提交

类模板中，若模板参数为非类型参数，通常需要指定其 具体 类型

得分：3.00

4. 已提交

在类中，若希望派生类能重写某函数，该函数应声明为 虚函数

得分：3.00

5. 已提交

定义函数模板时，通常使用 `template` 关键字，模板参数可以是 类型参数 或非类型参数

简答题

1.

说明 C++ 中基类析构函数为何需要声明为虚函数，并简述其作用。

5.00 得分：5.00

1.原因：如果基类的析构函数不是虚函数，那么当通过基类指针删除派生类对象时，只会调用基类的析构函数，而不会调用派生类的析构函数，导致资源泄漏。

2.作用：当基类析构函数声明为虚函数后，可以确保调用正确的析构函数。

2.

简述函数模板与普通函数的区别，并说明编译器如何处理函数模板的类型推导。

5.00 得分：5.00

1.两者区别：函数模板是泛型代码，编译器根据实际参数类型生成具体函数；而普通函数是针对特定类型的实现。

2.编译器如何处理函数模板的类型推导：编译器通过函数调用时的实参类型推导模板参数，生成对应的实例化函数。

1.

几何形状面积

【问题描述】

设计一个几何形状管理程序，包含抽象基类 Shape 和派生类 Circle 及 Rectangle。

Shape 包含纯虚函数 `area()`，计算面积。

Circle 包含半径（double），面积为  $\pi * radius^2$ （使用 3.141592653589793）。

Rectangle 包含宽和高（double），面积为 `width * height`。

【输入形式】

输入格式：

格式为 `<type> <param1> [<param2>]`，type 为 C（圆）或 R（矩形）。

圆：param1 为半径；矩形：param1 为宽，param2 为高，均为非负浮点数。

第一行：整数 n，表示形状数量。

接下来的 n

行，每行描述一个形状：

最后一行：整数 q，表示查询次数；随后 q 行，每行一个整数 index（0-based，查询第 index 个形状的面积）。

#### 【输出形式】

每个查询输出一行：Area of shape <index>: <area>，area 保留两位小数。

若 index 无效，输出 Area of shape <index>: -1.00。

#### 【样例输入】

```
4
C 2.0
R 2.0 3.0
C 1.0
R 4.0 5.0
3
0
2
5
```

#### 【样例输出】

```
Area of shape 0: 12.57
Area of shape 2: 3.14
Area of shape 5: -1.00
```

#### 【样例说明】

#### 【评分标准】

2.

向量运算

#### 【问题描述】

设计一个类模板 Vector2D，表示二维向量，包含 x 和 y 坐标（支持任意数值类型）。实现以下功能：

- 重载 + 运算符，返回两个向量坐标相加的结果。
- 重载 == 运算符，比较两个向量是否相等（浮点数比较允许误差  $1e-6$ ）。

#### 【输入形式】

第一行：整数 n，表示操作次数。

接下来的 n 行，每行格式为 <op> <x1> <y1> <x2> <y2>：

op 为 add（相加）或 equal（比较）。

x1, y1 为第一个向量坐标，x2, y2 为第二个向量坐标，均为浮点数。

#### 【输出形式】

对于 add，输出 Add: (<x>, <y>)，x 和 y 保留两位小数。

对于 equal，输出 Equal: <result>，result 为 true 或 false。

#### 【样例输入】

```
4
add 1.5 2.5 3.5 4.5
equal 1.0 2.0 1.0 2.0
```

equal 1.0 2.0 1.0 2.000001

equal 1.0 2.0 2.0 3.0

【样例输出】

Add: (5.00, 7.00)

Equal: true

Equal: true

Equal: false